

econcomplex

Technical Documentation — Version 0.1.0

Python Library for Economic Complexity
and Regional Science Indicators

Elton Freitas

Dependencies: `numpy` $\geq$ 1.21, `pandas` $\geq$ 1.3, `scipy` $\geq$ 1.7
Compatible with Python 3.9+

March 2026

Contents

1	Overview	3
1.1	Installation	3
1.2	Module Structure	3
2	Data Organisation	4
2.1	Standard Format: Long Table (<i>long format</i>)	4
2.2	Data Quality Rules	4
2.3	Column Names — <code>cols</code> Dictionary	4
2.4	Supported File Formats	5
2.5	Examples of Compatible Datasets	6
3	How to Run the Scripts	6
3.1	Prerequisites	6
3.2	Direct Execution via Terminal	6
3.3	Running in Jupyter Notebook / JupyterLab	7
3.4	Running in VS Code / PyCharm / Spyder	8
3.5	Minimal Execution Script	8
3.6	Running with RAIS / CNAE Data (real use case)	9
3.7	Running a Multi-Year Panel	9
4	Core Indicators (<code>core</code>)	10
4.1	Revealed Comparative Advantage (RCA)	10
4.2	Diversity and Ubiquity	10
5	Economic Complexity (<code>complexity</code>)	11
5.1	Eigenvector Method (Hidalgo & Hausmann, 2009)	11
5.2	Method of Reflections (Hidalgo & Hausmann, 2009)	11
5.3	Fitness-Complexity (Tacchini et al., 2012)	11
5.4	Subnational ECI	11
6	Product Space and Relatedness (<code>relatedness</code>)	11
6.1	Proximity (ϕ)	11
6.2	Density and Distance	12
6.3	Co-occurrence and Relatedness Indices	12
6.4	Cross-Space (relatedness between distinct classifications)	12
7	Regional Specialization (<code>specialization</code>)	12
8	Inequality and Concentration (<code>inequality</code>)	12
9	Productivity (<code>productivity</code>)	13
10	Patent-Based Complexity (<code>patents</code>)	13
11	Temporal Dynamics (<code>dynamics</code>)	13
12	Complexity Outlook (<code>outlook</code>)	13

13 Unified Pipeline (<code>compute_complexity</code>)	13
13.1 Function Signature	13
13.2 Parameters	14
14 Complete Examples	14
14.1 Example 1 — Regional Employment Analysis	14
14.2 Example 2 — Exports with COMEX Data	15
14.3 Example 3 — RAIS Time Panel (2010–2022)	15
15 References	16
16 License and Citation	16

Overview

econcomplex is a Python library that consolidates the best implementations of Economic Complexity and Regional Science indicators, combining functionalities from four reference packages in the literature:

- **EconGeo** (Balland, 2017) — R package focused on geographic and regional specialization indicators
- **economiccomplexity** (Vargas Sepúlveda, 2020) — R package with C++ backend (Armadillo) for complexity indicators
- **py-econcomplexity** (The Growth Lab at Harvard, 2020) — complete Python pipeline with time series support
- **py-economic-complexity** — modular Python implementation with Polars support and cross-space indicators

Installation

```

1 # From the library root directory
2 pip install -e .
3
4 # Or adding the path manually
5 import sys
6 sys.path.insert(0, "/path/to/econcomplex")
7 import econcomplex as ec

```

Listing 1: Installation via pip (local development)

Module Structure

Module	Contents
core	RCA, RPOP, Mcp, diversity, ubiquity, utilities
complexity	ECI/PCI (3 methods), subnational ECI
relatedness	Proximity, density, distance, co-occurrence, cross-space
specialization	LQ, Hachman, Krugman, specialization coef., similarity
inequality	Gini, locational Gini, Hoover-Gini, HHI, Shannon entropy
productivity	PRODY, EXPY, PGI, PEII
patents	Ease of Recombination, Modular Complexity
dynamics	Growth rates, entry/exit tracking
outlook	COI, COG (Complexity Outlook)
pipeline	<code>compute_complexity()</code> function — unified pipeline

Data Organisation

This section describes the input format expected by the library, ensuring that data is correctly structured before running any indicator.

Standard Format: Long Table (*long format*)

The **econcomplex** library works exclusively with **tables in long format** (*tidy data*), where each row represents a **unique combination** of region $\times$ activity (and period, when applicable):

region	activity	value	year (optional)
SP	cnae_10	12 345	2022
SP	cnae_25	6 789	2022
RJ	cnae_10	9 012	2022
RJ	cnae_25	3 456	2022
SP	cnae_10	13 200	2023
⋮	⋮	⋮	⋮

Warning

The library does **not** accept pivoted matrices (regions in rows, activities in columns). Use `pivot_to_matrix()` only for low-level functions that explicitly require the matrix.

Data Quality Rules

1. **No duplicate rows:** each (*region*, *activity*) pair must appear exactly once per period.
2. **Non-negative values:** the value column must contain only numbers ≥ 0 .
3. **No NaN:** remove or impute missing values before passing the DataFrame.
4. **Consistent granularity:** all regions must use the same geographic level (e.g., all municipalities *or* all states).
5. **Homogeneous activities:** use a single classification per analysis (e.g., CNAE 2-digit *or* CNAE 4-digit, not mixed).

Column Names — cols Dictionary

The pipeline uses a *cols* dictionary to map the actual column names of your DataFrame:

Key	Type	Description
"loc"	required	Region / location column
"act"	required	Activity / product / technology column
"val"	required	Numeric value column (jobs, exports ...)
"time"	optional	Period column (year, quarter ...)

```

1 import pandas as pd
2 import econcomplex as ec
3
4 df = pd.read_csv("my_data.csv")
5
6 # Check structure
7 print(df.dtypes)
8 print(df.shape)
9 print(df.isnull().sum())      # check for NaN
10 print(df.duplicated().sum())  # check for duplicates
11
12 # Basic cleaning
13 df = df.dropna(subset=["region", "cnae", "jobs"])
14 df = df[df["jobs"] >= 0]      # remove negative values
15 df = df.drop_duplicates(subset=["region", "cnae", "year"])
16
17 # Run the pipeline
18 result = ec.compute_complexity(
19     df,
20     cols={"loc": "region", "act": "cnae",
21          "val": "jobs", "time": "year"},
22 )

```

Listing 2: Verifying and preparing the DataFrame before the pipeline

Supported File Formats

Format	pandas reader	Notes
.csv	pd.read_csv()	Most common; use <code>sep=";"</code> for Brazilian CSVs
.parquet	pd.read_parquet()	Ideal for large datasets (RAIS, COMEX)
.xlsx	pd.read_excel()	Requires <code>openpyxl</code> ; avoid >100 000 rows
.feather	pd.read_feather()	High performance; requires <code>pyarrow</code>
.dta	pd.read_stata()	Stata files; preserves labels

```

1 import pandas as pd
2
3 # CSV with semicolon separator (Brazilian standard)
4 df = pd.read_csv("rais.csv", sep=";", encoding="latin-1",
5                  usecols=["municipality", "cnae2d", "year", "jobs"],
6                  dtype={"municipality": str, "cnae2d": str})
7
8 # Parquet (preferred for RAIS/COMEX)
9 df = pd.read_parquet("comex_ncm.parquet",
10                     columns=["co_mun", "co_ncm", "year", "vl_fob"])
11
12 # Excel
13 df = pd.read_excel("data.xlsx", sheet_name="Sheet1",
14                   usecols=["state", "sector", "year", "jobs"])

```

Listing 3: Reading different file formats

Examples of Compatible Datasets

RAIS/MTE

Formal employment by municipality $\times$ CNAE (2 or 4 digits) $\times$ year. Value column: active employment bonds on 31/12.

COMEX/MDIC

Exports by municipality $\times$ NCM $\times$ year. Value column: VL_FOB (value in USD).

INPI — Patents

Patent applications by State of the Federation $\times$ IPC classification $\times$ year.

Atlas of Economic Complexity (Harvard)

Exports by country $\times$ product (HS) $\times$ year. Available at <https://atlas.cid.harvard.edu/>.

Simulated data (testing)

Use `ec.make_sample_data(n_locs=20, nActs=50, seed=42)` to generate a synthetic DataFrame.

How to Run the Scripts

This section describes all the ways to run the **econcomplex** library, from direct execution of Python scripts to use in interactive environments such as Jupyter and IDEs.

Prerequisites

Warning

Make sure the dependencies are installed before running any script.

```
1 # Install main dependencies
2 pip install numpy pandas scipy
3
4 # Install the library in development mode
5 pip install -e /path/to/econcomplex
6
7 # Verify installation
8 python3 -c "import econcomplex as ec; print(ec.__version__)"
```

Listing 4: Installing the dependencies

Direct Execution via Terminal

The simplest way to use the library is to run a `.py` script directly in the terminal:

```
1 # Navigate to the project folder
2 cd "/path/to/1 - Pos-doc research/data/script"
3
4 # Run the full example script
5 python3 econcomplex/example_usage.py
6
7 # Run your own script
8 python3 my_script.py
```

Listing 5: Running scripts in the terminal

If the library is **not** installed via **pip**, add the path manually at the beginning of each script:

```

1 import sys
2 sys.path.insert(0, "/path/to/econcomplex")
3
4 import econcomplex as ec
5 print(ec.__version__)    # 0.1.0

```

Listing 6: Adding path to sys.path (without pip install)

Running in Jupyter Notebook / JupyterLab

```

1 cd "/path/to/1 - Pos-doc research/data/script"
2 jupyter notebook
3 # or
4 jupyter lab

```

Listing 7: Opening Jupyter from the scripts folder

Inside the notebook, the first cell should configure the path:

```

1 # Cell 1 -- configuration
2 import sys, os
3
4 BASE = os.path.expanduser(
5     "~/Library/CloudStorage/"
6     "OneDrive-Personnel/1 - Pos-doc research/data/script/econcomplex"
7 )
8 sys.path.insert(0, BASE)
9
10 import econcomplex as ec
11 import numpy as np
12 import pandas as pd
13
14 print("econcomplex", ec.__version__, "loaded successfully.")

```

Listing 8: Configuration cell in Jupyter

```

1 # Cell 2 -- load data
2 df = pd.read_csv("my_data.csv")    # columns: region, sector, jobs
3
4 # Cell 3 -- compute complexity
5 result = ec.compute_complexity(
6     df,
7     cols={"loc": "region", "act": "sector", "val": "jobs"},
8     method="eigenvector",
9     compute_coi_cog=True,
10 )
11 result.head()
12
13 # Cell 4 -- display ECI by region
14 eci = (result[["region", "eci"]]
15        .drop_duplicates()
16        .set_index("region")
17        .sort_values("eci", ascending=False))
18 print(eci)

```

Listing 9: Subsequent cells — normal usage

Running in VS Code / PyCharm / Spyder

In any IDE, configure the **Python interpreter** and the **working directory** to point to the scripts folder.

```

1 // .vscode/settings.json
2 {
3     "python.defaultInterpreterPath":
4         "/Library/Frameworks/Python.framework/Versions/3.12/bin/python3",
5     "terminal.integrated.cwd": "${workspaceFolder}"
6 }

```

Listing 10: Configure path in VS Code (settings.json)

In **Spyder**, go to *Tools* → *PYTHONPATH Manager* and add the `econcomplex/` directory.

Minimal Execution Script

Ready-to-use template for any CSV-format dataset:

```

1 """
2 Usage template for the econcomplex library.
3 Adapt the column names and the CSV file path.
4 """
5 import sys
6 sys.path.insert(0, "/path/to/econcomplex")
7
8 import pandas as pd
9 import econcomplex as ec
10
11 # 1. Load data
12 df = pd.read_csv("data.csv")
13 print("Columns:", df.columns.tolist())
14 print("Shape:", df.shape)
15
16 # 2. Compute all indicators
17 result = ec.compute_complexity(
18     df,
19     cols={
20         "loc": "region_column_name",    # e.g.: "municipality", "state"
21         "act": "activity_column_name",  # e.g.: "cnae", "product"
22         "val": "value_column_name",     # e.g.: "jobs", "exports"
23         "time": "year_column_name",     # optional
24     },
25     method="eigenvector",    # 'eigenvector' / 'reflections' / 'fitness'
26     threshold=1.0,
27     compute_coi_cog=True,
28 )
29
30 # 3. Export results
31 result.to_csv("complexity_results.csv", index=False)
32 print("Result saved to complexity_results.csv")
33 print("Generated columns:", sorted(result.columns.tolist()))
34
35 # 4. Individual indicators (optional)
36 mat = ec.pivot_to_matrix(df,
37                             "region_column_name",
38                             "activity_column_name",

```

```

39         "value_column_name")
40
41 eci, pci = ec.eci_pci(mat)
42 hachman = ec.hachman_index(mat)
43 krugman = ec.krugman_index(mat)
44 entropy = ec.shannon_entropy(mat)
45
46 output = pd.DataFrame({
47     "eci": eci, "hachman": hachman,
48     "krugman": krugman, "entropy": entropy,
49 })
50 print(output.round(3))

```

Listing 11: Minimal template — copy and adapt

Running with RAIS / CNAE Data (real use case)

```

1 import sys
2 sys.path.insert(0, "/path/to/econcomplex")
3
4 import pandas as pd
5 import econcomplex as ec
6
7 # Load aggregated RAIS
8 # Expected columns: municipality, cnae2d, year, jobs
9 rais = pd.read_parquet("rais_municipality_cnae.parquet")
10
11 # Filter a single year for analysis
12 rais_2022 = rais[rais["year"] == 2022].copy()
13
14 # Pipeline
15 result = ec.compute_complexity(
16     rais_2022,
17     cols={"loc": "municipality", "act": "cnae2d", "val": "jobs"},
18     method="eigenvector",
19     compute_coi_cog=False, # disable COG for large datasets
20 )
21
22 # Top 20 most complex municipalities
23 top20 = (result[["municipality", "eci"]]
24     .drop_duplicates()
25     .sort_values("eci", ascending=False)
26     .head(20))
27 print(top20.to_string(index=False))
28
29 result.to_csv("complexity_municipalities_2022.csv", index=False)

```

Listing 12: Usage with RAIS data — municipalities x CNAE

Running a Multi-Year Panel

To analyse the evolution of indicators over time, simply include the time column in the *cols* dictionary:

```

1 # rais with "year" column containing 2010, 2015, 2020
2 panel_result = ec.compute_complexity(
3     rais,

```

```

4     cols={
5         "loc": "municipality",
6         "act": "cnae2d",
7         "val": "jobs",
8         "time": "year",          # activates the temporal loop
9     },
10    method="reflections",
11    compute_coi_cog=False,
12 )
13
14 # The result already contains all years
15 print(panel_result["year"].value_counts().sort_index())
16
17 # Median ECI evolution by year
18 eci_trend = (panel_result[["year", "municipality", "eci"]]
19             .drop_duplicates()
20             .groupby("year")["eci"]
21             .median()
22             .round(4))
23 print(eci_trend)

```

Listing 13: Panel execution — multiple years

Note

The pipeline automatically iterates over each period, ensuring that the proximity matrix and complexity vectors are recalculated independently for each year.

Core Indicators (core)

Revealed Comparative Advantage (RCA)

Balassa's RCA is the main specialization filter:

$$RCA_{il} = \frac{x_{il} / \sum_l x_{il}}{\sum_i x_{il} / \sum_{il} x_{il}}$$

```

1 import econcomplex as ec
2 import pandas as pd
3
4 df = pd.read_csv("data.csv")
5 mat = ec.pivot_to_matrix(df, "region", "sector", "jobs")
6
7 rca      = ec.rca(mat)          # continuous values
8 mcp      = ec.mcp(mat, threshold=1) # 1 if RCA >= 1, 0 otherwise
9 rca_rpop = ec.rpop(mat)        # Puga & Turrini RCA (continuous)

```

Listing 14: Continuous and binary RCA

Diversity and Ubiquity

```

1 div = ec.diversity(mat) # number of sectors with RCA >= 1 per region
2 ubi = ec.ubiquity(mat)  # number of regions with RCA >= 1 per sector

```

Listing 15: Diversity and ubiquity

Economic Complexity (complexity)

Eigenvector Method (Hidalgo & Hausmann, 2009)

$$\text{ECI} = \frac{\vec{v}_2 - \mu(\vec{v}_2)}{\sigma(\vec{v}_2)}, \quad \text{PCI} = \frac{\vec{u}_2 - \mu(\vec{u}_2)}{\sigma(\vec{u}_2)}$$

where $\vec{v}_2$ is the second left eigenvector of matrix $\tilde{M}$:

$$\tilde{M}_{ij} = \frac{M_{ij}}{k_{c,0} \cdot k_{p,0}}$$

```
1 eci, pci = ec.eci_pci(mat, method="eigenvector")
2 print(eci.head())    # Series indexed by region
3 print(pci.head())    # Series indexed by product/sector
```

Listing 16: ECI and PCI using the eigenvector method

Method of Reflections (Hidalgo & Hausmann, 2009)

```
1 eci_r, pci_r = ec.eci_pci(mat, method="reflections", iterations=20)
```

Listing 17: Method of Reflections

Fitness-Complexity (Tacchini et al., 2012)

$$F_c^{(n+1)} = \sum_p M_{cp} Q_p^{(n)}, \quad Q_p^{(n+1)} = \frac{1}{\sum_c M_{cp} / F_c^{(n)}}$$

```
1 fit, comp = ec.eci_pci(mat, method="fitness", iterations=100)
```

Listing 18: Fitness-Complexity

Subnational ECI

```
1 # pci_national: PCI computed from national/international data
2 eci_sub = ec.subnational_eci(mat_sub, pci_national)
```

Listing 19: Subnational ECI with external PCI

Product Space and Relatedness (relatedness)

Proximity (ϕ)

$$\phi_{ij} = \min(P(\text{RCA}_i \geq 1 \mid \text{RCA}_j \geq 1), P(\text{RCA}_j \geq 1 \mid \text{RCA}_i \geq 1))$$

```
1 phi_d = ec.proximity(mat, continuous=False)    # based on binary Mcp
2 phi_c = ec.proximity(mat, continuous=True)     # based on continuous RCA
```

Listing 20: Discrete and continuous proximity

Density and Distance

$$\omega_{ij} = \frac{\sum_k M_{kj} \phi_{ij}}{\sum_k \phi_{ij}}, \quad d_{ij} = 1 - \omega_{ij}$$

```
1 density = ec.density(mat, phi_d)    # DataFrame loc x act
2 distance = ec.distance(mat, phi_d)  # DataFrame loc x act
```

Listing 21: Density and distance

Co-occurrence and Relatedness Indices

```
1 cooc      = ec.co_occurrence(mat)
2 phi_cos   = ec.cosine_proximity(mat)    # cosine-based
3 phi_cor   = ec.correlation_proximity(mat)
4 rca_rel   = ec.relatedness(mat, phi_d)  # relatedness densities
```

Listing 22: Co-occurrence and relatedness indices

Cross-Space (relatedness between distinct classifications)

```
1 # phi_cross: proximity between sectors and technologies (different
  # matrices)
2 phi_cross = ec.cross_space_proximity(mat_sector, mat_tech)
3 density_c = ec.density(mat_sector, phi_cross)
```

Listing 23: Cross-space proximity and relatedness

Regional Specialization (specialization)

```
1 lq          = ec.location_quotient(mat)    # equivalent to RCA
2 hachman     = ec.hachman_index(mat)        # 0 (diversified) to 1
  (specialized)
3 krugman     = ec.krugman_index(mat)        # structural difference between
  regions
4 coef_spec   = ec.spec_coefficient(mat)     # specialization coefficient
5 sim_exp     = ec.export_similarity(mat)    # productive structure
  similarity
```

Listing 24: Specialization indices

Inequality and Concentration (inequality)

```
1 gini_r      = ec.gini(mat)                # Gini of value distribution
2 gini_loc    = ec.locational_gini(mat)     # locational Gini (by sector)
3 hoover      = ec.hoover_gini(mat)         # Hoover index
4 hhi         = ec.hhi(mat)                 # Herfindahl-Hirschman
5 entropy     = ec.shannon_entropy(mat)     # Shannon entropy (by region)
```

Listing 25: Inequality indicators

Productivity (productivity)

```

1 prody = ec.prody(mat, gdp_per_capita) # income associated with the
    product
2 expy  = ec.expy(mat, prody)          # sophistication of the export
    basket
3 pgi   = ec.pgi(mat)                  # Product Gini Index
4 peii  = ec.peii(mat)                 # Product Export Inequality Index

```

Listing 26: Productivity and product inequality

Patent-Based Complexity (patents)

```

1 ease_rec = ec.ease_of_recombination(mat_pat)
2 mod_comp = ec.modular_complexity(mat_pat)

```

Listing 27: Patent indicators (input: patent x technology matrix)

Temporal Dynamics (dynamics)

```

1 growth = ec.growth_rates(df, "region", "sector", "jobs", "year")
2 entries = ec.entry_tracking(df, "region", "sector", "year")
3 exits  = ec.exit_tracking(df, "region", "sector", "year")

```

Listing 28: Growth rates and entry/exit tracking

Complexity Outlook (outlook)

The **COI** measures the latent diversification potential; the **COG** measures the expected complexity gain:

$$\text{COI}_c = \sum_p (1 - M_{cp}) \omega_{cp} Q_p$$

```

1 coi = ec.coi(mat, phi_d, pci)
2 cog = ec.cog(mat, phi_d, pci)

```

Listing 29: COI and COG

Unified Pipeline (compute_complexity)

Function Signature

```

1 def compute_complexity(
2     df: pd.DataFrame,
3     cols: dict,          # {"loc": str, "act": str, "val": str, "time":
        str}
4     method: str = "eigenvector", # 'eigenvector' / 'reflections' /
        'fitness'
5     threshold: float = 1.0,      # threshold for Mcp
6     compute_coi_cog: bool = True,
7     iterations: int = 20,        # for reflections/fitness

```

```

8 ) -> pd.DataFrame:
9     """
10    Returns the original DataFrame enriched with columns:
11    rca, mcp, diversity, ubiquity, eci, pci,
12    density, distance, [coi, cog]
13    """

```

Listing 30: Pipeline function signature

Parameters

Parameter	Type	Description
df	DataFrame	Table in long format
cols	dict	Column mapping (see Section 2.3)
method	str	Complexity method
threshold	float	Mcp threshold (default: 1.0)
compute_coi_cog	bool	Compute COI and COG
iterations	int	Iterations for iterative methods

Complete Examples

Example 1 — Regional Employment Analysis

```

1 import sys
2 sys.path.insert(0, "/path/to/econcomplex")
3
4 import pandas as pd
5 import numpy as np
6 import econcomplex as ec
7
8 # Generate synthetic data (or load a real CSV)
9 df = ec.make_sample_data(n_locs=50, n_acts=30, seed=42)
10 # Columns: loc, act, val
11
12 # Full pipeline
13 result = ec.compute_complexity(
14     df,
15     cols={"loc": "loc", "act": "act", "val": "val"},
16     method="eigenvector",
17     compute_coi_cog=True,
18 )
19
20 # Summary of generated indicators
21 print(result.columns.tolist())
22 # ['loc', 'act', 'val', 'rca', 'mcp', 'diversity', 'ubiquity',
23 #  'eci', 'pci', 'density', 'distance', 'coi', 'cog']
24
25 # Top 10 regions by ECI
26 top_eci = (result[["loc", "eci"]]
27             .drop_duplicates()
28             .sort_values("eci", ascending=False))

```

```

29         .head(10))
30 print(top_eci)
31
32 # Save
33 result.to_csv("regional_result.csv", index=False)

```

Listing 31: Complete regional employment analysis

Example 2 — Exports with COMEX Data

```

1 import sys
2 sys.path.insert(0, "/path/to/econcomplex")
3
4 import pandas as pd
5 import econcomplex as ec
6
7 # Load COMEX dataset aggregated by municipality x NCM x year
8 comex = pd.read_parquet("comex_mun_ncm.parquet")
9 comex_2023 = comex[comex["year"] == 2023].copy()
10
11 # Cleaning
12 comex_2023 = comex_2023.dropna(subset=["co_mun", "co_ncm", "vl_fob"])
13 comex_2023 = comex_2023[comex_2023["vl_fob"] > 0]
14
15 # Pipeline
16 result = ec.compute_complexity(
17     comex_2023,
18     cols={"loc": "co_mun", "act": "co_ncm", "val": "vl_fob"},
19     method="eigenvector",
20     compute_coi_cog=True,
21 )
22
23 # Save
24 result.to_csv("export_complexity_2023.csv", index=False)
25 print("Mean ECI:", result["eci"].dropna().mean().round(4))

```

Listing 32: Export analysis with COMEX/MDIC data

Example 3 — RAIS Time Panel (2010–2022)

```

1 import sys
2 sys.path.insert(0, "/path/to/econcomplex")
3
4 import pandas as pd
5 import econcomplex as ec
6
7 rais = pd.read_parquet("rais_municipality_cnae_2010_2022.parquet")
8
9 # Temporal pipeline -- iterates over each year automatically
10 panel = ec.compute_complexity(
11     rais,
12     cols={
13         "loc": "municipality",
14         "act": "cnae2d",
15         "val": "jobs",
16         "time": "year",
17     },

```



```

18     method="eigenvector",
19     compute_coi_cog=False,
20 )
21
22 # ECI median evolution
23 eci_trend = (panel[["year", "municipality", "eci"]]
24              .drop_duplicates()
25              .groupby("year")["eci"]
26              .median()
27              .round(4))
28 print(eci_trend)
29
30 panel.to_csv("panel_complexity_2010_2022.csv", index=False)

```

Listing 33: Pipeline with panel data — multiple periods

References

Balassa (1965)

Balassa, B. (1965). Trade liberalisation and “revealed” comparative advantage. *The Manchester School*, 33(2), 99–123.

Hidalgo & Hausmann (2009)

Hidalgo, C.A., & Hausmann, R. (2009). The building blocks of economic complexity. *PNAS*, 106(26), 10570–10575.

Balland (2017)

Balland, P.-A. (2017). *EconGeo: Computing Key Indicators of the Geography of Innovation*. R package.

Vargas Sepúlveda (2020)

Vargas Sepúlveda, M. (2020). *economiccomplexity: Computational Methods for Economic Complexity*. R package.

The Growth Lab at Harvard (2020)

The Growth Lab at Harvard University (2020). *py-econcomplexity: Economic Complexity in Python*. GitHub.

Tacchini et al. (2012)

Tacchini, E., Loreto, V., Pietronero, L. (2012). A New Metrics for Countries’ Fitness and Products’ Complexity. *Scientific Reports*, 2, 723.

License and Citation

This library is distributed under the MIT license. For use in academic work, we suggest citing the original packages in the versions corresponding to the indicators used.

```

1 import econcomplex as ec
2 print(ec.__version__)    # 0.1.0

```

Listing 34: Check the library version